

ECOM EXPERTS

Ecom layeredtable

Manual de usuario, tecnico y de desarrollo

ecom_layeredtable 1.0.0 · 07/09/2026

Contenido

¿Qué hace?

Instalación

El panel

Configuración (un solo formulario con pestañas)

Preguntas frecuentes

1. El problema que resuelve

2. Cómo se engancha (sin overrides)

3. Tablas propias y el ajuste a la tabla del módulo de facetas

4. El limpiador (classes/EcomLayeredtableCleaner.php)

5. Configuración (todas globales, prefijo ELT_)

6. Puntos de entrada

7. Limitaciones conocidas

Estructura (legacy, sin src/, sin Composer)

Convenciones que sigue

Cómo añadir una regla de limpieza

Cómo añadir un botón al panel

Prueba automática (sin tienda)

Cómo probarlo a mano

Desinstalación

¿Qué hace?

El módulo de navegación por facetas de PrestaShop (`ps_facetedsearch`) guarda en la tabla `layered_filter_block` una copia de cada listado de filtros que se muestra. Cada combinación de filtros, moneda, idioma y país es una copia nueva, y los robots de los buscadores prueban miles de combinaciones. Como ninguna copia caduca nunca, la tabla crece hasta ocupar cientos de MB o varios GB.

Ecom Layered Table mantiene esa tabla por debajo del tamaño que tú decidas (950 MB por defecto), borra lo que lleva tiempo sin usarse y evita que los robots y los rangos de precio la llenen. La tienda no cambia nada para el cliente: si un listado no está en la caché se calcula igual que siempre.

Instalación

1. Módulos → Subir un módulo → `ecom_layeredtable.zip` .
2. Configurar. El módulo ya funciona con los valores por defecto.
3. Opcional pero recomendado: copia la **URL del cron** que aparece al final del panel y programa una llamada cada hora en tu hosting. Con el cron activo puedes desactivar la ejecución automática desde la tienda (pestaña Automatización).

El panel

Arriba tienes cuatro tarjetas:

- **Datos vivos:** lo que ocupan de verdad las entradas de la caché, con la barra respecto al límite.
- **Fichero en disco:** lo que ocupa el fichero, incluido el espacio fragmentado que MySQL guarda tras los borrados. Solo baja con «Optimizar» (o al vaciar).
- **Entradas en caché:** número aproximado de entradas y cuántas conoce el módulo.
- **Última ejecución:** cuándo, desde dónde y cuánto borró.

Botones:

Botón	Qué hace
Limpiar ahora	Ejecuta todas las reglas (límite, fecha, uso, tamaño) en pasadas de unos segundos, una detrás de otra, hasta que la tabla está por debajo del límite. Si hay que borrar más del 60 % de la tabla, la vacía de golpe: es un segundo en vez de minutos, y la caché se regenera sola
Añadir columna de fecha	Solo aparece si la tabla aún no tiene fecha. La añade ahora aunque la tabla sea grande
Reconciliar	Da fecha y tamaño a las entradas que ya existían antes de instalar el módulo
Optimizar	<code>OPTIMIZE TABLE</code> : recupera el espacio fragmentado. Con tablas enormes puede tardar minutos
Vaciar la caché	Borra toda la caché. Se regenera sola. Pide confirmación
Reconstruir el índice de atributos / de precios	Lo mismo que hace el módulo de facetas en su propia página; útil para comprobar que los filtros siguen bien tras una limpieza
Ver registro / Borrar registro	El registro de depuración, si está activado

Debajo: las tablas `layered_*` de la tienda con su tamaño (el índice de precios también puede ser grande), el histórico de limpiezas y la URL del cron.

Configuración (un solo formulario con pestañas)

Límites - *Activar el guardián*: apágalo y el módulo no hace nada automático. - *Tamaño máximo de la tabla*: el límite duro. Por encima, se borran entradas hasta el *objetivo* (70 % por defecto: de 950 MB se baja a 665 MB). - *Número máximo de entradas y tamaño máximo de una entrada*: límites opcionales. - *Estrategia de evacuación*: qué sale primero cuando hay que hacer sitio. «Las usadas hace más tiempo» es la recomendada. - *Vaciar la tabla en vez de evacuar cuando haya que borrar más del N % (60)*: con una tabla de varios GB, borrar fila a fila tarda minutos; vaciarla tarda un segundo. - *Emergencia*: si el fichero en disco sigue por encima del límite y no se ha podido optimizar, se vacía la tabla. Es lo que garantiza que nunca pase de los 950 MB.

Fecha y uso - *Borrar las entradas con más de N días (7) y sin usar desde hace N días (3)*. - *Registrar el uso*: una consulta pequeña por página de listado. Necesaria para las reglas de uso. - *Optimizar*: recuperar disco automáticamente desde el back-office y el cron.

Prevención (lo que evita que la tabla crezca) - *Robots*: las peticiones de Googlebot, Bingbot, Semrush... no se guardan en caché. La lista se puede editar. - *Rangos de precio o peso*: no se guardan. Es la causa principal del crecimiento. - *Más de N filtros (3)*: las combinaciones de muchos filtros no se repiten; no merece la pena guardarlas. - *Tabla llena*: mientras esté por encima del límite no se guarda nada nuevo.

Automatización - Cada cuántos minutos puede lanzarse la limpieza automática, si se permite desde la tienda, y cuántos segundos puede durar cada ejecución según de dónde venga.

Registro - *Registro de depuración*: escribe en `modules/ecom_layeredtable/logs/` lo que decide el módulo en cada petición. Solo para diagnosticar; apágalo después.

Preguntas frecuentes

¿Puede romper los filtros? No. El módulo solo borra entradas de una caché que el módulo de facetas regenera sola. Los índices de precios y atributos no se tocan.

¿Por qué el fichero en disco no baja después de limpiar? MySQL no devuelve el espacio de las filas borradas hasta que se reconstruye la tabla. Pulsa «Optimizar» o deja que lo haga el cron.

Tengo varias tiendas. La tabla es la misma para todas, así que la configuración es global.

El panel dice que mi `ps_facetedsearch` no tiene el hook (versión 4.0.0 o anterior). Todo funciona igual salvo el contador de uso de cada entrada: la limpieza se guía por la antigüedad. Al actualizar el módulo de facetas a 4.0.1 o superior el contador se activa solo.

¿Qué es la «columna de fecha»? El módulo añade una fecha de creación a la propia tabla de la caché para poder limpiar por antigüedad. En servidores modernos es instantáneo; en MySQL antiguos con una tabla enorme se espera a que la tabla baje de 500 MB (o se fuerza con el botón).

El panel dice «optimización pendiente». La tienda ha detectado que el fichero supera el límite por fragmentación y no puede optimizar desde el front. Pulsa «Optimizar»; si no lo haces, tras un tiempo de gracia la tienda vaciará la tabla.

ecom_layeredtable — Documento técnico

Versión 1.0.0 · **PrestaShop** 8.0 a 9.x · **PHP** 7.4 a 8.4 · **ps_facetedsearch** 4.0.3, 4.0.4 y 5.0.0 **Autor** Ecom Experts · ecomyseo@gmail.com · Licencia AFL-3.0

1. El problema que resuelve

La tabla `prefix_layered_filter_block` es la caché de «bloques de filtros» del módulo de navegación por facetas. Cada fila guarda, serializado en PHP, el listado completo de facetas (nombres, URL, meta, recuentos de cada valor de cada atributo, característica, marca...) de una página de listado concreta.

Hecho verificado en el código	Consecuencia
Clave = <code>md5(tienda, moneda, idioma, controlador+id, país, serialize(filtros seleccionados))</code> (<code>src/Product/SearchProvider.php::generateCacheKeyForQuery</code>)	Cada combinación de filtros es una fila distinta; se multiplica por monedas, idiomas y países
Los rangos de precio y peso entran en el hash con sus valores exactos	Infinitas combinaciones: cada posición del deslizador o cada número escrito en la URL crea una fila
La tabla solo tiene <code>hash</code> y <code>data</code> (<code>ps_facetedsearch.php::installLayeredFilterTables</code>)	Sin fecha ni contador: ninguna fila caduca ni se puede saber cuál se usa
Lo único que la vacía es <code>invalidateLayeredFilterBlockCache()</code> = <code>TRUNCATE</code> , al guardar un producto, categoría, atributo o característica	Con catálogo estable crece para siempre; con catálogo muy editado se vacía entera cada vez
<code>data</code> es <code>TEXT</code> en 4.0.3 y en el 4.0.4 de PS 8.2.7; <code>LONGTEXT</code> desde el 4.0.4 de GitHub (<code>upgrade-4.0.4.php</code>) y en 5.0.0	Con <code>TEXT</code> los bloques de más de 64 KB se truncan en silencio y se regeneran en cada visita; con <code>LONGTEXT</code> una fila puede pesar varios MB
Se escribe con <code>REPLACE INTO</code> (borrado + inserción)	Fragmentación; InnoDB no devuelve el espacio al disco sin <code>OPTIMIZE TABLE</code> o <code>TRUNCATE</code>
Los robots recorren todas las URL con <code>?q=...</code>	Filas por combinaciones que ningún humano repite

Referencia pública: [PrestaShop/PrestaShop#17318](#).

2. Cómo se engancha (sin overrides)

Dos hooks, y los dos hacen lo mismo con distinta precisión:

- `actionProductSearchProviderRunQueryBefore` (núcleo, PS 1.7.6+, antes de cualquier listado). Recibe el `ProductSearchQuery`; los filtros seleccionados se leen de `getEncodedFacets()` (`Etiqueta-valor-valor/Etiqueta-símbolo-mín-máx`, con `\` de escape; un rango es una faceta de tres valores cuyos dos últimos son números). Es el respaldo para `ps_facetedsearch` 4.0.0 y anteriores, que **no tienen** el hook siguiente.
- `actionFacetedSearchCacheKeyGeneration` (`ps_facetedsearch` 4.0.1+), justo antes de leer o escribir la caché, con `filterKey` y `facetedSearchFilters` por referencia y el objeto `query`. Aquí los filtros son exactos y se puede calcular el mismo `md5`.

La decisión se toma una sola vez por petición (`self::$skipReason`). El módulo hace tres cosas desde ahí:

1. **Prevención.** Si la petición viene de un robot, lleva rango de precio o peso, lleva más de N valores de filtro o la tabla está por encima del límite, ejecuta `Configuration::set('PS_LAYERED_CACHE_ENABLED', 0)`. `Configuration::set()` escribe **solo en memoria** (`_new_cache_shop` / `_new_cache_global`, que es exactamente donde lee `Configuration::get()`, verificado en `classes/Configuration.php` de 8.2.7 y 9.1.1). El módulo de facetas comprueba esa clave en `Block::getFromCache()` y `Block::insertIntoCache()`, así que para esa petición ni lee ni escribe la caché. La base de datos no se toca y las facetas se calculan igual.
2. **Uso.** Recalcula el mismo `md5` que el módulo de facetas y anota `INSERT ... ON DUPLICATE KEY UPDATE hits = hits + 1, last_used = NOW()` en `prefix_ecom_layeredtable_meta`. Una consulta por página de listado cacheable (`category`, `manufacturer`, `supplier`). Se puede desactivar.
3. **Automático.** Si han pasado más de `ELT_AUTO_MINUTES` desde la última ejecución, lanza el limpiador con presupuesto `ELT_FRONT_BUDGET_SEC`.

`actionAdminControllerSetMedia` hace lo mismo desde el back-office con `ELT_ADMIN_BUDGET_SEC` y además carga CSS/JS del panel. El front controller `cron` (token propio) hace la pasada completa con `ELT_CRON_BUDGET_SEC`.

3. Tablas propias y el ajuste a la tabla del módulo de facetas

```
prefix_ecom_layeredtable_meta  hash CHAR(32) PK, date_add, last_used, hits, size  (índices en
last_used, date_add, size, hits)
prefix_ecom_layeredtable_log    id_log, date_add, trigger_type, action, rows_deleted, mb_before,
mb_after, rows_after, seconds, details
```

`size` es `LENGTH(data)` del bloque (bytes) y **solo se mide cuando una regla lo necesita** (estrategia `biggest`, `ELT_MAX_ROW_KB` o columna `TEXT`): leer el blob de cada fila es lo que costó 21 s sin borrar nada sobre una tabla real de 4,8 GB. Mientras no se conoce, el tamaño de una fila se estima con `AVG_ROW_LENGTH` de `information_schema`.

Columna `date_add` en `prefix_layered_filter_block`. El módulo la añade (`DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP` + índice). `ps_facetedsearch` escribe con `REPLACE INTO ... (hash, data)` nombrando las

columnas, así que la fecha se rellena sola y su `CREATE TABLE IF NOT EXISTS`, su `upgrade-4.0.4.php` y su `TRUNCATE` la respetan. Se intenta primero con `ALGORITHM=INSTANT` (MariaDB 10.3+, MySQL 8.0+); si el servidor no lo admite, solo se hace el `ALTER` con reconstrucción cuando la tabla ocupa menos de `ELT_ALTER_MAX_MB` (por defecto 500 MB), y hasta entonces la pantalla lo avisa. Al desinstalar la columna se queda (quitarla reconstruiría la tabla); es inocua.

4. El limpiador (`classes/EcomLayeredtableCleaner.php`)

`run()` ejecuta, por lotes de `ELT_BATCH_ROWS` y dentro del presupuesto de segundos, **en este orden**: lo que resuelve el problema va primero y siempre ejecuta al menos un lote; lo accesorio va al final con el tiempo que sobre.

Paso	Qué hace	Front	Admin/cron
Bloqueo	<code>GET_LOCK()</code> de MySQL (0 s de espera); si no está disponible, marca de tiempo en <code>Configuration</code>	sí	sí
Medida	<code>ANALYZE TABLE + information_schema.TABLES (DATA_LENGTH + INDEX_LENGTH = usado; DATA_FREE = fragmentado; la suma = fichero)</code>	sí	sí
0	Si la tabla está vacía (la vació el módulo de facetas), vacía la meta	sí	sí
1	Límite de tamaño. Si usado > <code>ELT_MAX_MB</code> : si hay que borrar más del <code>ELT_BULK_TRUNCATE_PCT</code> % de la tabla, <code>TRUNCATE</code> (1 s); si no, evacuar según estrategia hasta <code>ELT_TARGET_PCT</code> % (primer lote siempre, los demás con tiempo)	sí	sí
2	Exceso sobre <code>ELT_MAX_ROWS</code> , según estrategia	sí	sí
3	Columna <code>TEXT</code> : filas con <code>size >= 65535</code> (truncadas: <code>unserialize</code> falla y el módulo de facetas las reescribe en cada visita)	sí	sí
4	Filas con <code>size > ELT_MAX_ROW_KB</code>	sí	sí
5	TTL: <code>date_add < NOW() - ELT_TTL_DAYS</code> sobre la propia tabla si tiene la columna; si no, sobre la meta	sí	sí
6	Sin uso: <code>last_used < NOW() - ELT_UNUSED_DAYS</code>	sí	sí
7	Reconciliar: baja de meta huérfanos, alta de hashes sin meta (paginación por clave primaria, sin leer <code>data</code> ; fecha = <code>date_add</code> de la tabla) y, si una regla lo necesita, medir tamaños por lotes pequeños	no	sí, con el tiempo que quede
8	Si <code>DATA_FREE > ELT_OPTIMIZE_FREE_MB</code> o fichero > límite: <code>OPTIMIZE TABLE</code>	no	sí (si <code>ELT_OPTIMIZE</code>)
9	Si el fichero sigue > límite y <code>ELT_EMERGENCY_TRUNCATE : TRUNCATE</code> (desde el front, solo tras un periodo de gracia de $2 \times$ <code>ELT_AUTO_MINUTES</code> en el que la pantalla avisa «optimización pendiente»)	sí	sí
10	Añadir la columna <code>date_add</code> si aún falta y la tabla ya es pequeña	no	sí
Fin	Fila en <code>_log</code> , <code>ELT_LAST_STATS</code> (JSON), <code>ELT_LAST_RUN</code> , <code>continue</code> (sigue por encima del límite y se ha avanzado); liberar bloqueo	sí	sí

Estrategias (`ELT_STRATEGY`): `lru` (`last_used ASC`, `hits ASC`), `lfu` (`hits ASC`, `last_used ASC`), `oldest` (`date_add ASC`), `biggest` (`size DESC`). Sin meta (recién instalado), la evacuación toma filas de la propia tabla por clave primaria.

Los borrados se hacen siempre como `SELECT hash ... LIMIT lote + DELETE ... WHERE hash IN (...)` sobre las dos tablas, nunca con `DELETE ... JOIN ... ORDER BY` (no admite `LIMIT`).

El botón «Limpiar ahora» encadena peticiones de `ELT_ADMIN_BUDGET_SEC` segundos mientras el servidor devuelva `continue = true` (máximo 200 pasadas), mostrando el progreso.

5. Configuración (todas globales, prefijo `ELT_`)

Multitienda: la tabla es común a todas las tiendas, así que **todas** las claves se guardan con `Configuration::updateGlobalValue()` y la pantalla lo avisa cuando `Shop::isFeatureActive()`.

Clave	Por defecto	Pestaña
ELT_ENABLED	1	Límites
ELT_MAX_MB	950	Límites
ELT_TARGET_PCT	70	Límites
ELT_MAX_ROWS	0	Límites
ELT_MAX_ROW_KB	1024	Límites
ELT_STRATEGY	lru	Límites
ELT_BULK_TRUNCATE_PCT	60	Límites
ELT_EMERGENCY_TRUNCATE	1	Límites
ELT_TTL_DAYS	7	Fecha y uso
ELT_UNUSED_DAYS	3	Fecha y uso
ELT_TRACK_USAGE	1	Fecha y uso
ELT_OPTIMIZE	1	Fecha y uso
ELT_OPTIMIZE_FREE_MB	200	Fecha y uso
ELT_SKIP_BOTS / ELT_BOT_LIST	1 / lista de 25 firmas	Prevención
ELT_SKIP_RANGES	1	Prevención
ELT_MAX_FILTERS	3	Prevención
ELT_SKIP_WHEN_FULL	1	Prevención
ELT_AUTO_MINUTES	30	Automatización
ELT_AUTO_FRONT	1	Automatización
ELT_FRONT_BUDGET_SEC / ELT_ADMIN_BUDGET_SEC / ELT_CRON_BUDGET_SEC	3 / 20 / 120	Automatización
ELT_BATCH_ROWS	1000	Automatización
ELT_ALTER_MAX_MB	500	Automatización
ELT_DEBUG_LOG	0	Registro
ELT_LOG_KEEP	500	Registro

Internas: `ELT_CRON_TOKEN`, `ELT_LAST_RUN`, `ELT_LAST_STATS`, `ELT_LOCK`, `ELT_OPTIMIZE_PENDING_SINCE`, `ELT_FEATURES_VERSION`, `GMARTOS_*_ecom_layeredtable`.

6. Puntos de entrada

- **Panel:** `AdminModules&configure=ecom_layeredtable` → `getContent()` = alerta Gmartos + `addnewfeatures()` + guardado + `dashboard.tpl` + un `HelperForm` con cinco pestañas (`eltlimits`, `eltclean`, `eltprevent`, `eltauto`, `eltlog`) y un solo botón.
- **AJAX** (`AdminEcomLayeredtableController`, token de PrestaShop, POST `ajax=1&action=`): `stats`, `clean`, `reconcile`, `optimize`, `truncate`, `datecolumn`, `reindexAttributes`, `reindexPrices` (por `cursor`, reutiliza `Ps_Facetedsearch::fullPricesIndexProcess($cursor, true, true)`), `log` (`clear=1` borra). Cabeceras anti-caché sin `Content-Type`.
- **Cron:** `index.php?fc=module&module=ecom_layeredtable&controller=cron&token=...` (`&action=stats|optimize|truncate|reconcile` opcional). JSON.

7. Limitaciones conocidas

- `TABLE_ROWS` y `DATA_LENGTH` de InnoDB son estimaciones; el módulo lanza `ANALYZE TABLE` antes de medir para que sean fiables.
- Si otro módulo se engancha al mismo hook **después** y cambia `filterKey` o los filtros, el hash que calcula este módulo no coincide y esas filas quedan con `hits = 0` hasta que «reconciliar» las da de alta (y entonces se rigen solo por fecha).
- Con `ps_facetedsearch` ≤ 4.0.0 no hay hook de clave de caché: la prevención funciona por el hook del núcleo, pero no se registra el uso (`hits`), así que LRU/LFU equivalen a «más antigua primero». La pantalla lo avisa (detecta el hook leyendo `SearchProvider.php`).
- Pruebas: `utiles/ecom_layeredtable/prueba_sql.php` ejecuta el limpiador y los hooks con stubs de PrestaShop contra una MariaDB real (tabla sintética de 95 MB): vaciado en bloque, evacuación por lotes, presupuesto desde el front, TTL por columna, sin uso, truncadas, límite de filas, robots, rangos, N filtros, tabla llena, cadencia, bloqueo, desinstalación. Pasa con PHP 7.4 y 8.1. No se instaló en las tiendas locales (el usuario lo descartó).
- Desde el front nunca se ejecuta `OPTIMIZE` (puede tardar minutos); sin cron ni visitas al back-office, el espacio fragmentado se recupera con el vaciado de emergencia.
- El módulo no cambia el comportamiento de `ps_facetedsearch` de vaciar toda la caché al guardar un producto.

ecom_layeredtable — Documento de desarrollo

Estructura (legacy, sin `src/`, sin Composer)

```
ecom_layeredtable/
  ecom_layeredtable.php                                clase Ecom_Layeredtable (namespace global)
  classes/EcomLayeredtableCleaner.php                  medida, limpieza, optimize, truncate, histórico
  classes/EcomLayeredtableLogger.php                   registro en logs/ (solo con ELT_DEBUG_LOG)
  controllers/admin/AdminEcomLayeredtableController.php acciones AJAX del panel
  controllers/front/cron.php                           Ecom_layeredtableCronModuleFrontController
  views/css/admin.css · views/js/admin.js              panel (JS por delegación sobre document, fetch con
cache: 'no-store')
  views/templates/admin/dashboard.tpl                  panel
  views/templates/admin/gmartos_update.tpl             aviso de versión
  translations/es-ES/ModulesEcomlayeredtableAdmin.es-ES.xlf
  sql/install.sql · sql/uninstall.sql                  copia de referencia (el módulo ejecuta el SQL desde
PHP)
  logs/ (.htaccess Deny from all)
  index.php en todas las carpetas
```

Fuera del módulo: `manuales/ecom_layeredtable/` (estos documentos),
`utils/ecom_layeredtable/traduccion.es.json` (extracción y generación del `.xlf`),
`_specs/ecom_layeredtable/001-limpieza-cache/` (spec, plan, tareas, notas).

Convenciones que sigue

- `Module::trans()` reexpuesto como público en la clase del módulo (lo usan el controlador admin y las clases).
- `isUsingNewTranslationSystem() = true`; todas las cadenas con `d='Modules.Ecomlayeredtable.Admin'` o `$this->trans(..., array(), self::TRANS_DOMAIN)`. **Nunca** `mod=`.
- Claves de pestaña del `HelperForm` prefijadas (`eltlimits` ...) para no chocar con ids del DOM del back-office.
- Toda configuración es **global** (`getGlobalValue` / `updateGlobalValue`). `Configuration::updateValue()` ya escapa el valor: no se le pasa `pSQL()`.
- `foreach (is_array($x) ? $x : array() as ...)` en todo lo que venga de `executes()` o `glob()`.
- Migraciones en `addnewfeatures()` (tablas, hooks, tab, claves nuevas), guardadas por `ELT_FEATURES_VERSION`. Se llama en cada `getContent()`.
- Endpoints AJAX: `Cache-Control`, `Pragma`, `Expires` y **sin** `Content-Type`.
- `php -l` con PHP 7.4 y 8.1: sin `match`, sin propiedades tipadas, sin `str_contains`.

Cómo añadir una regla de limpieza

1. Añadir la clave a `Ecom_Layeredtable::$defaults` (aparece sola en `getConfig()`, `getConfigFormValues()`, `install()`, `uninstall()` y `addnewfeatures()`).
2. Añadir el campo en `renderForm()` (pestaña que toque) y su validación en `postProcess()`.
3. Añadir el paso en `EcomLayeredtableCleaner::run()` usando `deleteWhere($where, $orderBy, $limit, $label)`: la condición se escribe sobre la meta (`m.`), la función va por lotes, borra en las dos tablas, acumula `deletedBytes` y anota la acción en el histórico.
4. Traducir: `python utiles/ecom_layeredtable/traducciones.py listar`, añadir las cadenas a `es.json`, `generar`, `comprobar`.

Cómo añadir un botón al panel

1. Botón en `dashboard.tpl` con `data-elt-action="nombre"` (y `data-elt-confirm="clave"` si pide confirmación; la clave va en `elt_cfg.txt` desde `hookActionAdminControllerSetMedia`).
2. Método `ajaxProcessNombre()` en el controlador admin que termine en `$this->out(array(...) + $this->stats())`. El JS vuelve a pintar tarjetas, tablas e histórico con lo que devuelve `stats()`.

Prueba automática (sin tienda)

```
cd "Z:\modulos ps mios finalizados y ok\modulos IA\utiles\ecom_layeredtable"
set ELT_PORT=3399
set ELT_MODULE_DIR=Z:\modulos ps mios finalizados y ok\modulos IA\
C:\dev-tools\php\php-7.4\php.exe prueba_sql.php
C:\dev-tools\php\php-8.1\php.exe prueba_sql.php
```

`stubs.php` sustituye a `Db`, `Configuration`, `Tools`, `Module`, `Context` ... con lo justo para que el módulo corra contra la MariaDB portable (`python -c "import servir; servir.arrancar_db()"` en `C:\dev-tools\testshop`; se para con `taskkill /F /IM mysqld.exe`). Crea la base `elt_test`, la llena con filas sintéticas y comprueba cada regla y cada hook. Termina con `FALLOS: n`.

Cómo probarlo a mano

```
-- Llenar la tabla con filas sintéticas (solo en pruebas)
INSERT INTO ps_layered_filter_block (hash, data)
SELECT MD5(CONCAT(RAND(), a.n, b.n)), REPEAT('x', 20000)
FROM (SELECT 1 n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION
      SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10) a,
      (SELECT 1 n UNION SELECT 2 UNION SELECT 3 UNION SELECT 4 UNION SELECT 5 UNION SELECT 6 UNION
      SELECT 7 UNION SELECT 8 UNION SELECT 9 UNION SELECT 10) b;

-- Medir
SELECT TABLE_ROWS, ROUND((DATA_LENGTH+INDEX_LENGTH)/1048576,2) usado_mb,
ROUND(DATA_FREE/1048576,2) libre_mb
FROM information_schema.TABLES WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME =
'ps_layered_filter_block';
```

- Robot: abrir una categoría con `User-Agent: Googlebot` y comprobar que no aparece fila nueva.
- Rango: abrir `.../categoria?q=Precio-€-10-50` y comprobar lo mismo.
- Cron: `curl "<url del panel>"` devuelve un JSON con `summary.actions`.
- Activar el registro de depuración y navegar: `logs/elt-AAAA-MM-DD.log` dice por qué se saltó cada petición (`bot`, `range`, `too_many_filters`, `full`).

Desinstalación

Borra `prefix_ecom_layeredtable_meta`, `prefix_ecom_layeredtable_log`, la pestaña oculta y todas las claves `ELT_*` y `GMARTOS_*_ecom_layeredtable`. **No** toca `prefix_layered_filter_block` ni el resto de tablas del módulo de facetas.